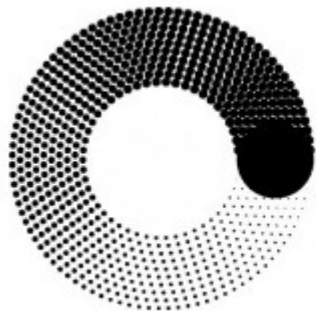




**МОСКОВСКИЙ
ПОЛИТЕХ**

**АВТОМАТИЗАЦИЯ ПРОЦЕССОВ ЖИЗНЕННОГО
ЦИКЛА ПРОГРАММНЫХ СРЕДСТВ**

**Лабораторная работа № 8.
Использование средств миграции баз данных**

**Москва
2024**

Теоретические основы

Непрерывная интеграция базы данных

Непрерывная интеграция базы данных — это быстрая интеграция изменений схемы и логики базы данных в процесс разработки приложений и предоставление немедленной обратной связи разработчикам по любым возникающим проблемам.

Непрерывная доставка базы данных

Непрерывная доставка базы данных — это процесс, направленный на внесение изменений в развернутую базу данных для реализации новых функций, обновлений и исправлений ошибок как можно быстрее и безопаснее.

Это достигается за счет оптимизации процесса разработки и сохранения кода приложения и кода базы данных в готовом к развертыванию состоянии.

Чтобы быстрее выводить на рынок новое программное обеспечение, компании в последнее десятилетие пытались модернизировать способы создания и доставки программного обеспечения. В своих усилиях по модернизации и улучшению конвейера непрерывной доставки многие инструменты сосредоточились на внедрении CI и CD в процесс изменения кода приложения.

Однако этого не произошло с базами данных. Многие организации по-прежнему полагаются на собственную команду поддержки базы данных, которая вручную проверяет и развертывает изменения БД. Учитывая, что опыт конечного пользователя неполный без полного стека программного обеспечения, который включает в себя как приложение, так и базу данных, растет интерес и потребность в средствах непрерывной интеграции и доставки для базы данных.

Цель инструментов непрерывной интеграции базы данных состоит в том, чтобы привнести в базу данных одни и те же передовые методы интеграции и развертывания, а также обеспечить прохождение кода SQL через конвейер выпуска программного обеспечения, синхронизированного с кодом приложения. Согласовывая развертывания баз данных с развертываниями приложений, команды получают гораздо большую отдачу от своих инвестиций в инструменты и обновления процессов. Это, в свою очередь, помогает командам быстрее и качественнее выводить на рынок новые инновации.

Лучшие практики при интеграции CI/CD для базы данных

Многие передовые методы, применимые к CI приложений и средствам доставки, легко применимы и к CI базы данных. Эти передовые методы включают в себя:

1. Отслеживание изменений кода базы данных

Изменения в коде базы данных должны отслеживаться в той же системе управления исходным кодом или версиями, что и код приложения. Код базы данных не следует обрабатывать отдельно или отслеживать в совершенно другой системе. Отдельная система для кода базы данных приводит к излишним усилиям, отсутствию наглядности, путанице и ошибкам. Код приложения и базы данных также начнет дрейфовать в отдельной системе и смещаться.

2. Автоматическая проверка кода базы данных и обратная связь

Как только код приложения попадает в репозиторий, сразу же запускается серия автоматических тестов. Эти тесты оценивают наличие проблем в коде, требующих доработки. К сожалению, это не относится к коду базы данных в большинстве компаний. Одной из самых больших проблем при ускорении развертывания баз данных является избавление от ручной проверки кода SQL, которую должны выполнять администраторы баз данных.

Эта утомительная ручная работа может и должна быть в значительной степени устранена с помощью интеллектуальной автоматизации. Таким образом разработчики могут получить немедленную обратную связь по коду SQL — так же, как они делают это с кодом приложения — и избежать длительного состояния ожидания, в котором они переходят к другой задаче. Эта неэффективность обратной связи вызывает большие задержки в развертывании баз данных и способствует снижению качества. Это также заставляет разработчиков переключаться между контекстами, чтобы вносить исправления в изменения кода SQL, которые они написали несколько дней или недель назад.

3. Упаковка кода базы данных

Важная часть философии DevOps — «создать один раз, развертывать часто». По сути, все инструменты непрерывной интеграции позволяют встраивать код приложения в неизменяемый пакет для последовательного, воспроизводимого и предсказуемого последующего развертывания. Если что-то пойдет не так, то его ошибки сразу же можно отследить либо в коде приложения, либо в среде при работе с неизменяемым артефактом. Инструменты непрерывной интеграции базы данных должны приносить те же самые улучшения в код базы данных.

4. Обеспечение видимости состояния базы данных

Еще один ключевой принцип DevOps — «усиливать обратную связь». Решения для непрерывной интеграции нацелены на быстрое обеспечение наглядности и обратной связи. Таким образом, решение для непрерывной интеграции базы данных должно иметь доступный интерфейс. Этот интерфейс должен позволить всем заинтересованным сторонам быстро понять состояние каждой базы данных. Кроме того, важно, чтобы решение интегрировалось с системами тикетов, такими как JIRA и TFS, и поддерживало стратегии параллельной разработки, которые обычно используются в корпоративных командах разработчиков.

Задания

1. Создать в репозитории проекта директорию для хранения актуальной схемы базы данных.
2. Провести анализ существующих инструментов статического анализа безопасности SQL-кода, выбрать наиболее удобное, по мнению команды, средство, обосновать выбор.
3. Добавить в CI/CD пайплайны для всех веток шаг, осуществляющий проверку безопасности SQL файлов
4. Внести изменение в код ПО и базу данных, таким образом, изменив актуальную схему БД.
5. Выгрузить при помощи штатных средств СУБД актуальную схему БД и сохранить ее в созданной директории, добавить ее к проекту и зафиксировать изменение коммитом.
6. После успешного тестирования нового функционала на TEST-стенде, вручную внести изменения в схему БД на STAGE-стенде перед развертыванием обновления.
7. Развернуть обновление на STAGE-стенде (при помощи коммита в DEV-ветку).
8. Проверить корректную работу примененной схемы БД и нового обновления приложения.
9. Провести анализ существующих инструментов автоматизации миграции схемы БД в CI/CD пайплайне, выбрать наиболее удобное, по мнению команды, средство, обосновать выбор.
10. Внедрить в CI/CD пайплайн для DEV-ветки шаг(и), осуществляющий автоматическое развертывание новой схемы БД на STAGE-сервере путем предварительной сверки схем БД на TEST-сервере и STAGE-сервере

11. Протестировать работу нового шага в пайплайне путем изменения схемы БД и созданием коммита в DEV-ветке, убедиться, что схемы БД идентичны

12. Создать отдельный пайплайн, идентичный пайплайну для DEV-ветки, но осуществляющий доставку и развертывание на PROD-сервере. Схема БД в данном случае применяется для соответствия STAGE-серверу

13. В созданный пайплайн добавить шаг, осуществляющий резервное копирование информации из БД перед внесением изменений

14. Проверить работу нового пайплайна, продемонстрировать успешное сохранение резервной копии БД

15. Подготовить отчет по лабораторной работе (текстовый документ и скриншоты с описанием по вышеописанным пунктам).